

Team Charter / Team Contract

Team Name	Team 4		
Project Name	Local Produce Exchange		
Course	ICS613	Semester	Summer 2026

1. Team Purpose

Team 4 is building an online local produce exchange web application called “Surplus”. This produce exchange is intended to serve an invite-only community by helping them share extra produce and other food before it goes to waste. The basic idea is this: if one person has too many tomatoes and another person could use them, the app should make that exchange easy.

We are also using this project as an opportunity to practice our software engineering skills as a team via the software development lifecycle. That means we will gather requirements, design the system, build features, test our work, and deliver working software through regular milestones.

2. Project Scope and Objectives

Project Vision

Local Produce Exchange is an invite-only web application where people in a trusted community can post extra food items and claim items from one another. The goal is to turn food that might have been wasted into food that gets used.

Goals

Add 3–5 measurable or concrete project goals.

- **Reproducible deployment:** Make sure teammates on a clean machine can follow the deployment guide and get the app running in under 30 minutes, with seeded demo data.
- **Maintain clear documentation:** Produce and maintain comprehensive project documentation including requirements, system designs, user stories, meeting notes to support future development and team communication.
- **Deliver working end to end Demonstration:** Effectively demonstrate a complete produce exchange portal by walking through the full use case of posters and claimers through the following states: Requested, Approved, Picked Up, Completed.
- **Maintain testing threshold:** Ensure with edge cases and error path testing

In Scope

List the features, responsibilities, or deliverables the team is committing to.

- Invite-only registration and login, plus member profiles
- Browse, search, and filter active listings
- Member dashboard that shows active listings, incoming and outgoing requests, and exchange history
- Request Queue to ensure requests are handled in order without conflicts
- Full claim lifecycle: REQUESTED, APPROVED, PICKED_UP, COMPLETED, DENIED, and CANCELLED

- Database schema, supporting single source of truth for listings, threads, users, reviews, etc
- Private message thread for each exchange (poster -> recipient)
- Ratings and reviews after an exchange is completed
- Photo uploads on listings
- An admin role
- Dietary Restrictions Tags for posts
- Key status change notifications

Out of Scope

List major items the team is intentionally not taking on.

- Payments, prices, tips, or any paid transactions. This exchange is free.
- Public sign-up. Access is invite-only.
- “Looking for” posts - posters only offer up what they have
- Production-grade scaling, load handling, and security hardening. This is a class project, not a commercial product.
- Customer support
- Multi-language support
- Mobile app
- Delivery or shipping

3. Team Members and Roles

List each member, their role, and the responsibilities they will primarily own.

Team Member	Role	Primary Responsibilities	Secondary Responsibilities
Kennan Kaneshiro	Team Member	Write backend code (Python, SQL, TypeScript) to connect application frontend to backend server and database.	Creating and organizing documentation of the project for maintainable development.
Shea Stevens	DBA	Database design and administration; schema ownership	Front end system design
Matt Ong	QA lead	Test planning, bug tracking	Design contributor and Test tool reviewer
Kim Cates	[Fill in]	[Fill in]	[Fill in]
Jeff Wu	Team Member	Set up and maintain the dev environment and npm scripts for easy, cross-platform local builds; Github Actions to lint each PR	Write Typescript to implement front end functionality and Python for backend functionality as needed for any User Story

4. Team Values

Choose 4–6 values that describe how the team wants to work together.

- **Talk early and honestly.** If something is confusing, blocked, broken, or bigger than expected, we say so early and help each other figure it out.

- **Accountability.** Given the team's busy schedules, keeping deliverables on schedule helps others keep their requirements on track as well.
- **Flexibility.** On the flip side of the coin, remember that things do come up, and working together as a team to fix issues as they occur is equally important.
- **Share Knowledge.** If you figure something out, or are skilled in a certain concept, help others to achieve a better understanding together.
- **Fairness.** Share workload and support each team member with respect

5. Communication Plan

Document the tools, response expectations, and meeting cadence the team will use.

Purpose	Tool / Channel	Notes
Quick team communication	Discord team project channel	Main place for project communication. A pinned message holds the GitHub repo URL and Google Drive shared folder links.
Task tracking	GitHub Issues and Projects board	Every story, task, and bug is an issue, assigned to someone, grouped by milestone, and tracked on the board.
Code repository	GitHub	One shared repo. We use branches, pull requests, and reviews before merging to main.
Meeting notes	Google Docs shared folder	Notes for every meeting, including decisions, action items, and owners.
Documentation and notes	Google Docs / Google Drive	Charter, requirements, and design docs start here. Final versions are committed to or linked from the repo for grading.

6. Working Agreements

Describe expectations for participation, reliability, responsiveness, and workload sharing.

- **Keep commitments realistic:** we each take a fair share of work for each milestone, keep our GitHub issues current, and speak up early if there is a problem or something might slip.
- **Get Merge Approval:** Have at least one other team member review your code before merging your code
- **Be Responsive:** Use the discord server as a single point of contact, make sure everyone is on the same page as we work through requirements.

7. Decision-Making Process

Explain how the team will make technical, scope, and scheduling decisions.

- **Start with consensus:** most of our decisions should come from talking through it in a meeting or on Discord until the team agrees.

- **Technical decisions:** proposed in a GitHub issue or meeting. If the team cannot agree, the lead for that area makes the call and records the reason.
- **Scope decisions:** Proposed in a meeting or GitHub issue. Changes that add or remove features require the whole team to weigh in. Discord messages can be used to raise concepts.
- **Scheduling decisions:** Due to working schedule, we'll most likely be meeting in the evening or on the weekends. We should do our best to review the milestone timeline, and address any deadline risks or blockers to reprioritize if necessary.
- **Tie breakers:** if a vote splits two to two, the Team Lead breaks the tie and records the decision and reason.

8. Development Workflow

Describe how work will be tracked, how branches and pull requests will be used, and what "done" means.

- **Work tracking:** every task, user story, and bug gets a GitHub issue with an owner and a spot on the Projects board: To Do, In Progress, In Review, or Done.
- **Git Branches:** Work should be done on git branches corresponding to its GitHub Issue
- **Code Reviews:** When code should be reviewed (2-3 approvals?) before it is merged to the master branch.
- **Definition of Done:** Completed functionality after code review of implementation and verification it passes QA tests, merged with master and updated with appropriate commit message

9. Quality Standards

Define your expectations for code quality, testing, documentation, and deployment readiness.

- **Continuous integration:** a GitHub Actions workflow runs our linters on every pull request, Ruff for the Python code and ESLint for the TypeScript code. These checks must pass before a pull request can merge, which protects main alongside reviews and branch protection.
- **Testing:** Every business rule (claim quantity validation, status transitions, permission checks) must have at least one automated unit test before the feature is considered done.
- **Code coverage:** Aim for meaningful coverage on backend business logic; coverage reports are generated and reviewed at each milestone.
- **Clear set up and transparency:** README, setup instructions, documentation, and comments

10. Project Artifacts

List the project artifacts the team commits to maintaining during the course.

- **Team and process:** team charter; GitHub repository with README; protected main branch; pinned Discord repo link; project board, issues, and milestones; meeting notes and decision log; retrospective and defect-triage notes.
- **Requirements:** high-level use cases; 25 to 30 user stories with acceptance criteria; domain model; cross-team requirements review packet and report.
- **Code and quality:** manual test cases and GitHub Actions CI workflows.
- **Documentation:** Project documentation that details how the system is designed, features, database schemas, etc. for easy management and future development.
- **Deployment:** Dockerfile and docker-compose configuration, environment variable documentation, and a validated step-by-step deployment guide with seeded demo data.
- **Testing:** Automated unit test suite covering business rules and permission checks; QA packet with manual test cases tied to user stories including edge cases and negative paths.

11. Conflict Resolution and Accountability

Explain what the team will do if someone is unresponsive, misses work, or if a conflict arises.

- **Talk to each other first:** if there is a disagreement or someone is falling behind, the people involved talk directly and respectfully as early as possible.
- **Handle silence or missed work quickly:** if a member goes quiet or misses a commitment, the team reaches out on Discord. If there is no response within two days, the team reassigns the at-risk work to keep the project moving and records it in the meeting log.
- **Separate the problem from the person:** Feedback is about the work, not the individual.
- **Have clear Deliverables:** Ensure task is explicitly defined to prevent miscommunication

12. Risks and Mitigation

Identify likely risks and write how the team will reduce or manage them.

- **Someone becomes unavailable.** Mitigation: use secondary owners for every area, distribute issues fairly each milestone, and follow the escalation steps in Section 11.
- **Scope creep.** Mitigation: any new feature idea goes through the full team before being added; out-of-scope items are logged but deferred.
- **Integration issues between frontend and backend.** Mitigation: agree on API contracts early and document them in the design doc before implementation begins.

13. Milestones and Timeline

Capture major checkpoints and target dates.

Milestone	Target Date	Owner(s)	Notes
Team Formation (repo, board, Discord)	June 1, 2026	Team	
Team Charter	June 5, 2026	Team	
Inception Presentation	June 8, 2026	Team	Presentations on June 9, approximately 10 minutes total
Core Requirements Artifacts	June 8, 2026	Jeff	
Cross-Team Requirements Review Packet	June 15, 2026	Shea	
Technical Design Document	July 1, 2026	Team	
R1 Demo + Presentation	July 6, 2026	Team	Presentations on July 7; retrospective and defect triage July 9
Manual Test Cases / QA Packet	July 14, 2026	Shea, Matt	
R2 Demo + Presentation	July 27, 2026	Team	Presentations on July 28
Deployment Guide + Release Readiness	August 6, 2026	Team	

Final Submission + Final Presentation	August 11, 2026	Team	Final demos August 11 to 13; peer evaluation and reflection August 13
---------------------------------------	-----------------	------	---

14. Team Commitment Statement

We, the members of this team, agree to contribute consistently, communicate professionally, and support one another in completing this project successfully. We understand that this course evaluates not only the final software product, but also the engineering process used to create it. We therefore commit to maintaining strong collaboration, clear documentation, responsible project management, and high standards of technical work throughout the semester.

15. Signatures

Each member should acknowledge the charter with a name and date.

Name	Signature / Acknowledgment	Date
Kennan Kaneshiro	Kennan Kaneshiro	06/05/2026
Shea Stevens	<i>Shea Stevens</i>	6/5/2026
Matt Ong	<i>Matt Ong</i>	6/5/2026
Kim Cates	[Fill in]	[Fill in]
Jeff Wu	<i>Jeff Wu</i>	6/5/2026